

บทที่ 2

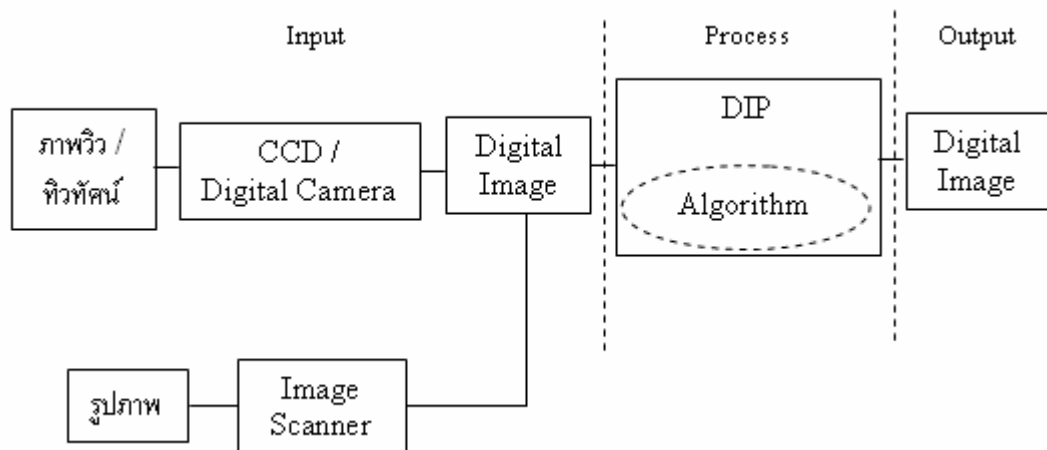
การประมวลผลภาพ

2.1. บทนำ

โดยปกติแล้วสายตาของบุคคลทั่วไปจะมองเห็นภาพทิวทัศน์ต่างๆ เป็นลักษณะแบบอนาล็อก ซึ่งสามารถอธิบายได้ด้วยคณิตศาสตร์ ที่มีตัวแปรแบบนับได้อย่างต่อเนื่อง แต่สำหรับเครื่องคอมพิวเตอร์ นั้นจะใช้เลขฐานสองเป็นหลักสำหรับการคำนวณ เมื่อนำภาพมาแปลงเข้าสู่เครื่องคอมพิวเตอร์ ภาพนั้นก็จะกลายเป็น ภาพดิจิทัล (Digital Image)

2.2. กระบวนการประมวลผลภาพดิจิทัล

การประมวลผลภาพดิจิทัล ได้มาจากภาพทิวทัศน์ต่างๆ แล้วถูกถ่ายภาพนั้นๆ ออกมาด้วยกล้องดิจิทัล หรือสแกนเนอร์ ก็จะได้ภาพดิจิทัลออกมา แต่ภาพที่มานั้นเป็นภาพดิจิทัลที่ยังไม่ได้ผ่านกระบวนการ การประมวลผลภาพ (Digital Image Processing) เมื่อผ่านกระบวนการการประมวลผลภาพ แล้วจึงได้ภาพดิจิทัลที่เป็นภาพดิจิทัล ดังนั้น เราสามารถแสดงภาพตั้งแต่การนำภาพวิว ทิวทัศน์ จนถึงเอาต์พุต ดังรูปที่ 2.2.1



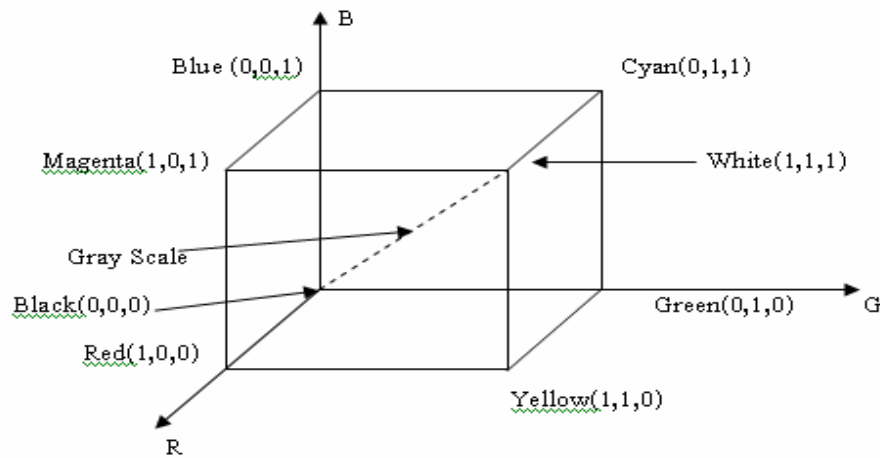
รูปที่ 2.1 กระบวนการประมวลผลภาพดิจิทัล

2.3. โมเดลสี

โมเดลสี ประกอบด้วย 3 แม่สีหลัก ได้แก่ สีแดง เขียว และน้ำเงิน ถ้านำแต่ละแม่สีมาพล็อตกราฟในระบบพิกัด Color Space โดยแต่ละสีมีค่า 0 ถึง 1 (0 แสดงถึงสีดำ และ 1 แสดงถึงสี

ขาว) จะได้ภาพการผสมสีทางแสงหรือการบวกแม่สีเข้าด้วยกัน (Additive Primary Color) ดังรูปที่

2.2



รูปที่ 2.2 โมเดลสี RGB

ถ้าแต่ละแม่สีมีขนาด 8 บิต แม่สี 3 แม่สีรวมกันก็จะมีขนาดเท่ากับ 24 บิต ซึ่งสามารถสร้างสีใหม่ได้ถึง $256 \times 256 \times 256$ เท่ากับ 16,777,216 สี ดังนั้นภาพสีขนาด 24 บิต จะมีค่าสีของพิกเซลอยู่ในช่วงที่ประกอบด้วย

R ระดับ 0 จนถึง 255 ($0 \leq R \leq 255$)

G ระดับ 0 จนถึง 255 ($0 \leq G \leq 255$)

และ B ระดับ 0 จนถึง 255 ($0 \leq B \leq 255$)

ในบางครั้งถ้าต้องการแปลงโมเดลสี ให้เป็น Gray Scale จะใช้สมการ

$$\text{Gray Scale} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \quad (2.1)$$

แต่เราสามารถใช้อีกสมการ โดยการหาค่าเฉลี่ยทั้งสามสีดังนี้

$$\text{Gray Scale} = (R + G + B) / 3 \quad (2.2)$$

จากรูปที่ 2.2 ค่า Gray Scale ก็คือค่าที่อยู่ในช่วง (0,0,0) จนถึง (1,1,1)

2.4. ไฟล์ข้อมูลภาพชนิดวินโดว์บิตแมป

2.4.1. โครงสร้างของไฟล์ข้อมูลภาพชนิดวินโดว์บิตแมป

โครงสร้างของไฟล์ข้อมูลภาพชนิดวินโดว์บิตแมป จะประกอบด้วย 3 ส่วน คือ

(i) ข้อมูลเฮดเดอร์ คือข้อมูลที่อยู่บริเวณส่วนหัวของไฟล์ ซึ่งจะประกอบไปด้วยข้อมูลที่บอกรายละเอียดต่างๆของภาพ เช่น ความกว้าง ความยาวของภาพ จำนวนสี จำนวนบิต ความละเอียด เป็นต้น

(ii) ข้อมูลงานสี คือข้อมูลที่บอกถึงชุดของงานสี ที่เกิดจากการผสมแม่สีทั้งสาม คือ แดง เขียว และน้ำเงิน มาผสมกัน ได้เป็นสีต่างๆ ตามจำนวนสีของภาพ เช่น รูปขนาด 4 บิต จะมี

16 ระดับสี รูปขนาด 8 บิต จะมีขนาด 256 ระดับสี เป็นต้น ซึ่งถ้ามีจำนวนสีน้อยๆ ก็จะมีการเก็บค่าจากสีนี้ลงไฟล์ไปด้วย แต่ถ้าเป็นรูปประเภท 24 บิตจะไม่มีค่าจากสี แต่จะใช้วิธีการเก็บค่าแม่สีทั้งสามลงไปเป็นข้อมูลแทนเพราะถ้าเก็บค่าจากสี ที่มีถึง 16.7 ล้านสีลงไปด้วยจะเปลืองพื้นที่มาก ข้อแตกต่างที่สำคัญของบิตแมปขนาดนี้ คือ ไฟล์บิตแมป จะเก็บค่าของงานสี ชุดละ 4 ไบต์ แต่ก็ใช้แค่ 3 ไบต์ คือ แดง เขียว และน้ำเงิน อย่างละ 1 ไบต์

(iii) ข้อมูลภาพ คือ ข้อมูลสีของภาพแต่ละจุดที่มาประกอบกันเป็นรูปภาพ ซึ่งค่าที่เก็บจะเป็นค่าที่ใช้ในการชี้ตาราง หมายเลขอะไร เช่น จุดแรกมีค่าเป็น 10 ก็ให้ไปเปิดตารางหมายเลข 10 สมมติว่า ค่าสีของแม่สีเป็น $R = 0$ $G = 0$ และ $B = 100$ ก็จะได้จุดนี้เป็นสีน้ำเงิน ซึ่งถ้าเป็นในกรณีของรูป 24 บิต จะเป็นการอ่านข้อมูลขึ้นมา 3 ค่า เป็นค่าของแม่สี RGB แล้วนำไปผสมบนจอภาพแทน

2.4.2. การจัดเก็บไฟล์ข้อมูลชนิดบิตแมป

การเก็บไฟล์ข้อมูลภาพชนิดบิตแมป มีการเก็บอยู่ 2 แบบ คือ

(i) แบบเข้ารหัสข้อมูล

- RLE 4 เป็นการบีบอัดข้อมูลแบบ Run – length Encoder แบบ 4 บิต
- RLE 8 เป็นการบีบอัดข้อมูลแบบ Run – length Encoder แบบ 8 บิต

(ii) แบบไม่ได้เข้ารหัสข้อมูล

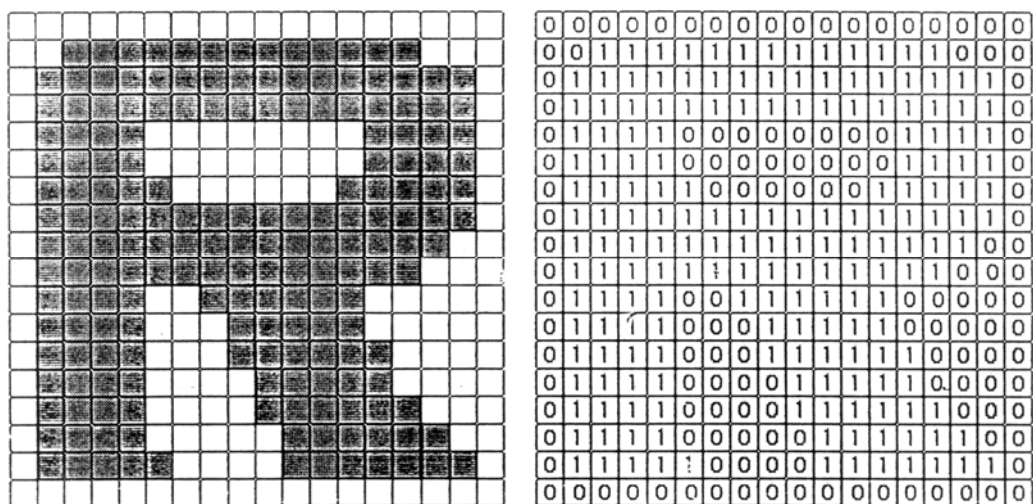
เป็นการเก็บข้อมูลจริงของสีของพิกเซล ซึ่งทำให้ขนาดของไฟล์ค่อนข้างใหญ่ แต่จะทำการแสดงผลได้รวดเร็วกว่า เพราะไม่ต้องเสียเวลาในการถอดรหัสข้อมูล

2.5. การสร้างภาพไบนารี

อุปกรณ์ที่มีความสามารถในการแสดงผลได้แค่ 2 ระดับ หรือ 2 สี คือ สีขาวกับสีดำยังมีการใช้กันอย่างแพร่หลาย เช่น เครื่องพิมพ์ (Printer) เครื่องโทรสาร (Fax) จอภาพแสดงผลแบบโมนโครม (Monochrome Monitor) เป็นต้น

จะเห็นได้ว่าการที่เราจะแก้ปัญหาการแสดงผลภาพ ที่มีความเข้มหลายระดับบนอุปกรณ์ที่สามารถแสดงผลได้ 2 ระดับนั้น จะต้องทำการแปลงข้อมูลภาพให้เป็นภาพไบนารีก่อน (Binary Image) ซึ่งการสร้างภาพไบนารี นั้นก็หมายถึงการแปลงข้อมูลภาพที่มีระดับความเข้มหลายระดับให้เป็นภาพที่มีระดับ (Multi Level Image) ความเข้มเพียง 2 ระดับ นั่นคือ 1 จุด ภาพมี 2 ค่าเท่านั้น คือ 0 กับ 1 โดยจุดภาพที่แทนด้วย 1 จะหมายถึงจุดภาพที่มีสีดำ ส่วนจุดที่แทนด้วย 0 จะหมายถึงจุดภาพที่มีสีขาว เมื่อทำการแปลงเป็นภาพไบนารีแล้วจึงนำภาพนั้นไปแสดงผลบนอุปกรณ์เหล่านั้น จะเห็นได้ว่าการแปลงข้อมูลภาพหลายระดับเป็นภาพไบนารี จึงมีความจำเป็นและมีประโยชน์มากในการแสดงผลภาพ ที่มีระดับความเข้มของภาพหลายระดับบนอุปกรณ์ ที่มีความสามารถในการแสดงผลได้ 2 ระดับ สำหรับประโยชน์อีกประการหนึ่งในการแปลงข้อมูลภาพเป็นภาพไบนารีคือการลดเนื้อที่การเก็บข้อมูล ภาพจะใช้เนื้อที่ในการเก็บ 8 บิต เมื่อ

สร้างเป็นภาพไบนารีแล้วสามารถลดลงได้ถึง 8 เท่า นั่นคือ 1 จุดภาพจะใช้เนื้อที่ในการเก็บ 1 บิต อีกทั้งยังสามารถนำไปประยุกต์ใช้งานได้อย่างแพร่หลาย เช่นนำไปประยุกต์ใช้ในการวิเคราะห์เอกสารในขั้นตอนที่เรียกว่า การประมวลผลขั้นต้น (Preprocessing) เป็นต้น



รูปที่ 2.3 การสร้างภาพไบนารี

ในการสร้างภาพไบนารีนั้น สามารถทำได้ด้วยใช้เทคนิคการทำเทรชโฮล (Thresholding Technique) โดยจะพิจารณาว่าจุดภาพนั้น ควรจะเป็นจุดสีขาวหรือจุดสีดำ ซึ่งจะกระทำโดยการเปรียบเทียบระหว่างจุดภาพเริ่มต้นกับค่าคงที่ค่าหนึ่งซึ่งเรียกว่า ค่าเทรชโฮล (Threshold Value) เทคนิคนี้ใช้กันมากในกรณีที่ข้อมูลภาพมีลักษณะแตกต่างกันระหว่างวัตถุ (Object) และพื้นหลัง (Background) โดยค่าของจุดภาพใดๆ ที่มีค่ามากกว่า หรือเท่ากับค่าเทรชโฮลจะถูกเปลี่ยนให้เป็น 0 (จุดขาว) ซึ่งการทำงานสามารถแสดงได้ดังสมการที่ 2.3

$$t(x, y) = \begin{cases} 1 & ; f(x, y) < thr \\ 0 & ; f(x, y) \geq thr \end{cases} \quad (2.3)$$

เมื่อ $f(x, y)$ คือ ข้อมูลภาพที่เป็นพิกเซล

$t(x, y)$ คือ ข้อมูลภาพที่เป็นพิกเซล ที่ถูกแปลงเป็นสีขาวและดำ

thr คือ ค่าเทรชโฮล เป็นค่าคงที่ ที่ใช้กำหนดเป็นสีขาวและสีดำ

0 คือ จุดสีขาว

1 คือ จุดสีดำ

ในการสร้างภาพไบนารี โดยใช้เทคนิคเทรชโฮลเพื่อให้ได้ผลลัพธ์ที่เหมาะสมและคมชัด สิ่งที่สำคัญที่สุดคือ ค่าเทรชโฮล เนื่องจากถ้าเลือกค่าเทรชโฮลที่ไม่เหมาะสม (ค่าเทรชโฮลที่มีค่าน้อยเกินไปหรือมากเกินไป) ภาพที่ได้อาจจะไม่เหมาะสม ขาดความคมชัดและรายละเอียดบางส่วนขาดหายไป กล่าวคือภาพที่ได้อาจจะมืดเกินไป (จุดดำมากเกินไป) หรือสว่างเกินไป (จุดขาวมากเกินไป) หรือภาพที่ได้มีสิ่งรบกวน (Noise) เกิดขึ้น อันเป็นผลทำให้ภาพผลลัพธ์ที่ได้ไม่สวยงามเท่าที่ควร ดังนั้นปัญหาของการสร้างภาพไบนารีโดยวิธีเทรชโฮลนี้คือ ทำอย่างไรจึงจะสามารถคำนวณหาค่าเทรชโฮลที่เหมาะสมสำหรับแต่ละภาพที่จะนำมาทำการสร้างภาพไบนารี ซึ่งมีวิธีการคำนวณหาค่าเทรชโฮลหลายวิธี โดยแต่ละวิธีเหมาะสมกับลักษณะการทำงานที่แตกต่างกันไป เช่นการหาค่าเทรชโฮลโดยการกำหนดค่าล่วงหน้า (Preassigned Threshold Value) การหาค่าเทรชโฮล จากค่ากลาง (Mid – Range Threshold Value) แต่ละวิธีอธิบายได้ดังนี้

2.5.1. การหาค่าเทรชโฮลโดยการกำหนดค่าล่วงหน้า (Preassigned Threshold Value) การหาค่าเทรชโฮลโดยวิธีการกำหนดค่าล่วงหน้านี้เป็นวิธีที่ง่ายที่สุด ซึ่งจะเป็นการคำนวณหาค่าเทรชโฮลโดยการกำหนดเองจากผู้ใช้งาน ซึ่งการกำหนดนี้จะขึ้นอยู่กับประสบการณ์ของผู้ใช้นั้นๆ โดยการเลือกค่าคงที่ค่าหนึ่ง ซึ่งเรียกค่านั้นว่า ค่าเทรชโฮล โดยค่าที่เลือกมานี้จะเป็นค่าที่อยู่ระหว่างค่าต่ำสุดและค่าสูงสุดของระดับ ความเข้มของข้อมูลภาพอินพุต เช่น ภาพข้อมูลอินพุตมีเกรย์สเกล 256 ระดับ จะมีเกรย์สเกลได้ตั้งแต่ 0-255 เมื่อเลือกค่าเทรชโฮลได้แล้ว สามารถสร้างภาพไบนารีได้โดยสมการ 2.5.1.1 ซึ่งเป็นสมการที่ใช้หาค่าเทรชโฮลโดยการคำนวณจากค่าเฉลี่ยเลขคณิต

$$thr = \frac{\sum_{i=0}^{m \times n} f(x_i, y_i)}{m \times n} \quad (2.4)$$

เมื่อ thr คือ ค่าเทรชโฮล

$$\sum_{i=0}^{m \times n} f(x_i, y_i) \text{ คือ ผลรวมของค่าเกรย์สเกล ทุกพิกเซล}$$

m x n คือ ขนาดของมิติ หรือขนาดของรูป

2.5.2. การหาค่าเทรชโฮลจากค่ากลาง (Mid – Range Threshold Value) การหาค่าเทรชโฮลโดยพิจารณาจากค่ากลาง เป็นการหาค่าเทรชโฮลที่แตกต่างจากการหาค่าเทรชโฮลวิธีแรก สำหรับวิธีนี้จะเป็นการคำนวณหาค่าเทรชโฮลโดยอัตโนมัติโดยไม่ต้องให้ผู้ใช้งานเป็นผู้กำหนด โดยการหาค่าเทรชโฮลวิธีนี้ได้อาศัยการคำนวณพื้นฐานทางสถิติในเรื่องของการหาค่ากลางหรือค่าเฉลี่ย (Mean) มาประยุกต์ใช้ ค่าเทรชโฮล ที่คำนวณได้จะเป็นค่าที่ได้จากค่ากึ่งกลางที่อยู่ระหว่างค่าระดับความเข้มสูงสุด (Maximum Level) และค่าระดับความเข้มต่ำสุด (Minimum Level) ของข้อมูลภาพอินพุต สำหรับการคำนวณค่ากึ่งกลางนี้สามารถคำนวณได้จากสมการที่ 2.5

$$thr = \frac{Max(f(x,y)) + Min(f(x,y))}{2} \quad (2.5)$$

เมื่อ thr คือ ค่าเทรชโฮล

Max(f(x,y)) คือ ค่าสูงสุดของค่าเกรย์สเกล

Min(f(x,y)) คือ ค่าต่ำสุดของค่าเกรย์สเกล

2.6. การแยกอักขระออกจากภาพ

กระบวนการสำคัญอีกขั้นตอนหนึ่ง ในการประมวลผลภาพเบื้องต้นก่อนที่จะไปสู่ขั้นตอนการจดจำรูปแบบ ก็คือกระบวนการแยกวัตถุออกจากพื้นหลัง ซึ่งในขั้นตอนนี้จะเป็นการแยกข้อมูลภาพที่เป็นตัวอักษรออกจากข้อมูลภาพทั้งหมด โดยแยกออกมาทีละตัวอักษรเพื่อไปเข้าสู่กระบวนการจดจำรูปแบบซึ่งสามารถประมวลผลได้ที่ละหนึ่งตัวอักษรเท่านั้น

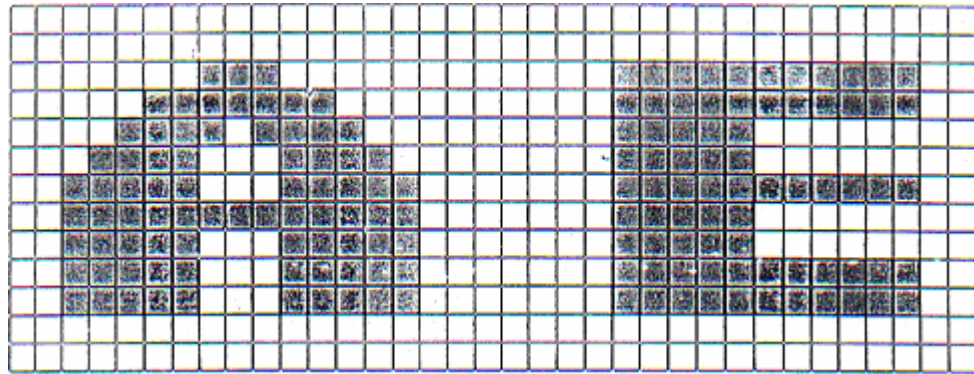
2.6.1. การหารอบตัวอักขระโดยวิธี Line Crossing

เมื่อรับข้อมูลภาพที่ได้จากการเปลี่ยนข้อมูลเป็นรูปแบบไบนารี ที่มีค่า 0 กับ 1 เรียบร้อยแล้วซึ่งข้อมูล 0 จะเป็น ส่วนที่เป็นพื้นหลังและ 1 แทนส่วนที่เป็นตัวอักษร หลักการเบื้องต้นคือการหาค่าพิกเซลที่เป็น 0 ที่ต่อเนื่องกันตลอดทั้งแนวตั้ง และแนวนอนทำให้ได้ขนาดของกรอบ (Block) ข้อมูลภาพวัตถุที่มีขนาดต่าง ๆ กัน จากนั้นก็จะทำการพิจารณาเลือกขนาดของกรอบที่ต้องการจากความแตกต่างของจำนวนพิกเซล ความสูง ความกว้าง และตำแหน่ง เป็นต้น ซึ่งก็จะได้กรอบของตัวอักษรที่ต้องการ

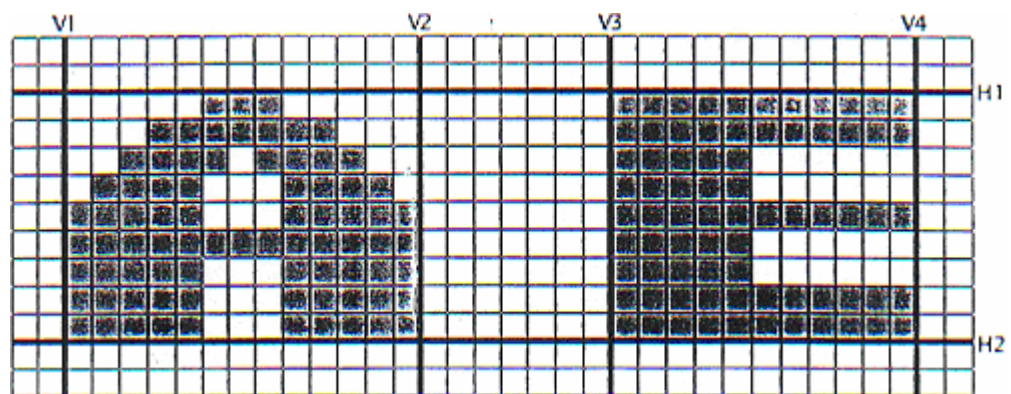
ตัวอย่างการแยกข้อมูลภาพที่มีตัวอักษร 2 ตัว เพื่อให้เห็นแนวทางในกระบวนการแยกเป็นขั้นตอนง่ายๆ ดังนี้ จากข้อมูลของภาพ ดังรูปที่ 2.6.1.1 จะพิจารณาค่าพิกเซลที่เป็น 0 ที่ต่อเนื่องตลอดระยะทางระหว่างระยะขอบเขตระยะด้านบนและระยะด้านล่างของแต่ละบรรทัด (จะทำการสแกนตามแนวตั้ง) แล้วหาขอบเขตระยะด้านหน้าและด้านหลังของแต่ละตัวอักษร (สแกนตามแนวนอน) เป็นขอบเขตของแต่ละตัวอักษร เพื่อใช้ในการประมวลผลต่อไป ดังแสดงในรูปที่ 2.5 จะได้ระยะ V1, V2, V3, V4 เป็นขอบเขตของระยะด้านหน้าและด้านหลังของตัวอักษร และจะได้ระยะ H1, H2 เป็นขอบเขตของระยะด้านบนและระยะด้านล่างของตัวอักษรตามลำดับ นั่นคือ เมื่อเราพิจารณาขนาด ความกว้าง และความสูง ในช่วงที่ยอมรับ ซึ่งเป็นคุณสมบัติของตัวอักษรที่ต้องการ ก็จะสามารถแยกตัวอักษรที่ต้องการจากข้อมูลภาพทั้งหมดได้ดังรูปที่ 2.6.1.3

ข้อดี ของการทำ Line Crossing คือ ง่ายต่อการเขียนโปรแกรม และมีประสิทธิภาพในการแยกอักขระกับแบบอักขระที่มีข้อผิดพลาดน้อย ตัวอักขระไม่ติดกันมาก และถ้าเป็นการ segment ตัวอักขระภาษาอังกฤษจะได้ผลลัพธ์ที่ดีกว่าตัวอักขระภาษาไทย

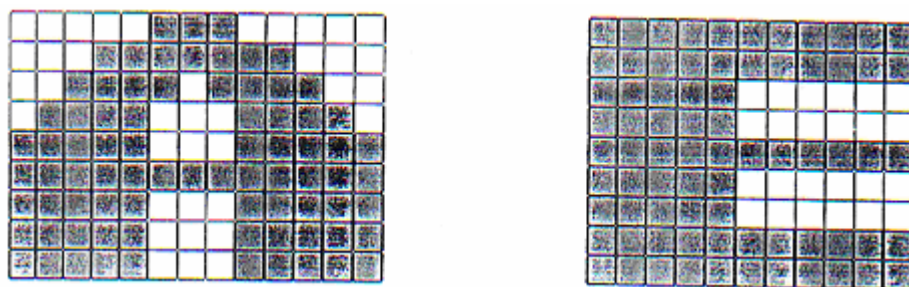
ข้อเสีย การทำการแยกอักขระที่มีการเหลื่อมล้ำกันจะไม่สามารถใช้เทคนิคในการแยกนี้ได้



รูปที่ 2.4 ข้อมูลภาพไบนารี



รูปที่ 2.5 ข้อมูลภาพไบนารีที่ถูกสแกนตามแนวนอนและแนวตั้ง



รูปที่ 2.6 ข้อมูลภาพไบนารีที่ถูกแยกวัตถุออกจากภาพ

2.6.2. เทคนิคการตามรอยขอบภาพ (Contour Following)

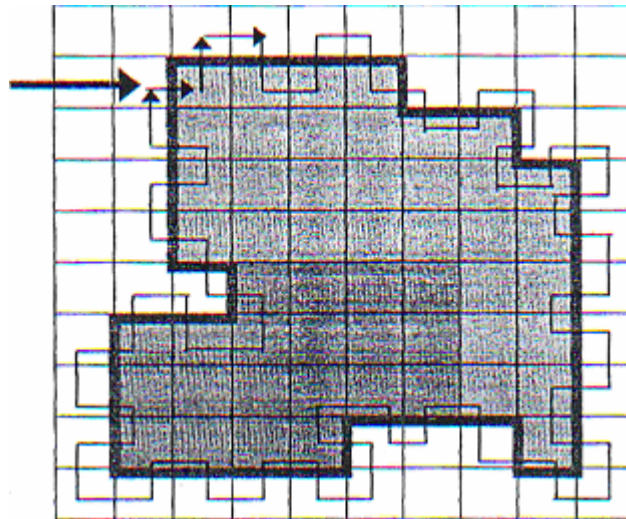
เทคนิคการตามรอยขอบภาพจะช่วยให้เราทำการ Segment ตัวอักษรได้ง่ายขึ้น โดยไม่ต้องคำนึงถึงรูปร่างของวัตถุที่จะทำการ Segment เมื่อพิจารณาขอบของวัตถุจะมีค่าที่มีความ

แตกต่างกันกับสีของพื้นหลัง เช่น ภาพตัวอักษรสีดำที่วางอยู่บนพื้นกระดาษสีขาว การทำงานของเทคนิคนี้ในการทำ Contour Following ตามรูปที่ 2.7 ทำได้ตามขั้นตอนต่อไปนี้

1. จุดภาพปัจจุบันมีข้อมูลเป็นสีดำ ให้เลี้ยวซ้ายจากทิศทางปัจจุบัน
2. จุดภาพปัจจุบันมีข้อมูลเป็นสีขาว ให้เลี้ยว ให้เลี้ยวขวาจากทิศทางปัจจุบัน
3. สิ้นสุดเมื่อจุดภาพปัจจุบันอยู่ตำแหน่งเดียวกับจุดเริ่มต้นพอดิ

ข้อดีของการทำวิธี Contour Following ก็คือ เราไม่ต้องสนใจว่ารูปร่างของตัวอักษรจะมีลักษณะเป็นอย่างไร เมื่อเราวิ่งไปตามขอบของภาพ เราก็จะได้ขอบของตัวอักษรออกมา

ข้อเสียของการทำวิธีการนี้ก็คือ ถ้าอักษรมีส่วนที่ติดกันอยู่ก็จะไม่สามารถที่จะใช้วิธีนี้ในการแยกตัวอักษรออกจากกันได้



รูปที่ 2.7 แสดงวิธีการตรวจหาขอบของภาพด้วยวิธีการ Contour Following

2.6.3. เทคนิคการตามรอยขอบภาพ ด้วยเมทริกซ์ (Contour with matrix)

เทคนิคนี้คล้ายกับการทำ Contour Following แต่จะต่างกันตรงที่เทคนิคนี้จะมีการดึงเอาข้อมูลภาพที่อยู่ในขอบเขตของการทำ Contour ไปใช้ด้วย ทำให้เราได้ตัวอักษรทั้งตัวไปใช้งาน และการใช้งานไม่เพียงแต่ดูว่าข้อมูลตรงจุดนั้นเป็น 0 หรือ 1 แต่จะใช้ matrix ช่วยตรวจสอบแทน matrix ที่ใช้ขนาด 3×3 ดังรูปที่ 2.8 แนวคิดของการทำ Contour with matrix ดังรูปที่ 2.9 มีขั้นตอนการทำงานดังต่อไปนี้

1. ให้กำหนดจุดดำที่พบเป็นจุดกลางของ matrix(P5) คัดลอกจุดนี้ลงใน buffer แล้วลบจุดกลางนั้นออกจากรูปภาพ

2. หาจุดกลางถัดไป โดยตรวจสอบข้อมูลรอบๆ จุดกลางปัจจุบัน(P5) ซึ่งเป็นจุดสีดำ คือ ข้อมูลตัวอักษร การตรวจสอบจะเริ่มจากด้านบนซ้ายก่อน คือ ตำแหน่ง P1,P2,P3,P4,P5, P6,P7,P8 และ P9 ตามลำดับ กำหนดจุดดำที่พบจุดแรกให้เป็นจุดกึ่งกลาง matrix

3. เมื่อรอบๆ จุดกลุ่มนั้นไม่มีจุดดำแล้ว จะทำการ return กลับไปยังจุดกลางตัวก่อน

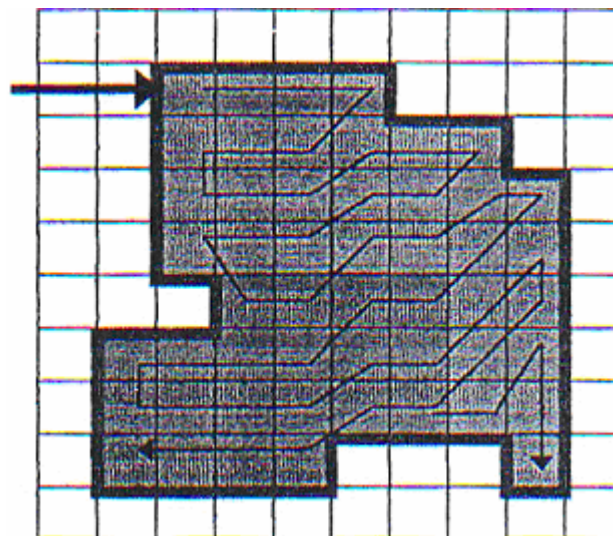
4. การทำงานจะเป็นเช่นนี้ไปเรื่อยๆ จนกระทั่งไม่มีจุดดำรอบจุดกลางทุกตัว

ข้อดีของวิธี Contour with matrix ก็คือ สามารถดึงเอาส่วนที่เป็นเนื้อตัวอักขระออกมาได้ทั้งหมด ซึ่งเป็นประโยชน์ต่อการนำไปใช้ในกระบวนการอื่นๆ ที่เนื้อของข้อมูลนั้นเป็นสิ่งจำเป็น

ข้อเสียของวิธีนี้ก็คือ เมื่อเราต้องดึงเอาข้อมูลทุกส่วนของตัวอักขระออกมาก็จะทำให้เราต้องใช้เวลาในการคัดลอก และลบข้อมูล วิธีการนี้ทำงานแบบ recursive ถ้ามีการ backtracking มากเกินไปอาจทำให้ stack overflow ได้

P1	P2	P3
P4	P5	P6
P7	P8	P9

รูปที่ 2.8 เมทริกซ์ขนาด 3 x 3



รูปที่ 2.9 แสดงการหาขอบโดยวิธี Contour with matrix

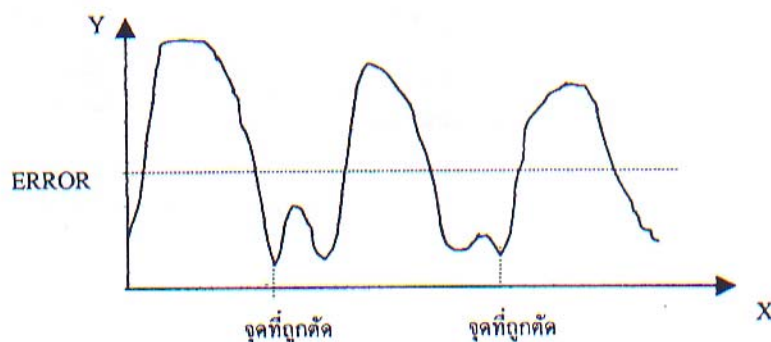
2.6.4. เทคนิคการแยกตัวอักขระที่ติดกันโดยใช้ Histogram

Histogram จะเป็นการพล็อตกราฟการนับความถี่ที่เกิดขึ้นซ้ำๆ กันในแต่ละระดับ คือ แกนตั้ง (Y) จะเป็นค่าความถี่ที่เกิดขึ้นซ้ำๆ กัน และแกนนอน (X) จะเป็นค่าระดับความเข้มของพิกเซล มีค่าตั้งแต่ 0 ถึง 255 ในการพล็อตกราฟ

Histogram ใช้เพื่อวัดความหนาแน่นของข้อมูลภาพในช่วงที่กำหนด โดยค่าความหนาแน่นที่ใช้เป็นจำนวนจุดดำ เพื่อช่วยในการวิเคราะห์ภาพตัวอักขระ โดยเก็บเป็นค่าเปอร์เซ็นต์

Error สำหรับนำไปใช้เปรียบเทียบกับตัวอักษรแต่ละตัว เพื่อหาตัวอักษรที่คาดว่าจะมีการติดกันอยู่ และการใช้ Histogram จะช่วยให้โปรแกรมสามารถตัดสินใจได้ว่า ควรที่จะตัดส่วนที่ติดกันของอักขระที่ตรงส่วนไหน เพื่อให้สามารถแยกตัวอักษรที่ติดกัน ให้ออกมาเป็นตัวอักษรเดี่ยวให้ได้มากที่สุด รูปที่ 2.10 แสดงการเลือกจุดที่จะทำการแยกตัวอักษรออกจากกัน การแยกส่วนที่ติดกันของตัวอักษร โดยใช้ histogram มีวิธีการดังนี้

1. ทำการหาค่า Histogram ของตัวอักษรที่สามารถแยกออกมาจากรูปภาพได้
2. กำหนดค่าเปอร์เซ็นต์ Error เพื่อใช้ในการแยกตัวอักษรออกจากกัน
3. ทำการแยกตัวอักษรตรงส่วนที่มีค่าความหนาแน่นเท่ากับ หรือต่ำกว่าค่าเปอร์เซ็นต์ Error ที่ได้กำหนดไว้ โดยจะเลือกแยกตัวอักษรตรงส่วนที่มีค่าความหนาแน่นต่ำที่สุด
4. ถ้าค่าความหนาแน่นที่ต่ำที่สุดมีหลายตำแหน่ง จะใช้ตำแหน่งที่อยู่ตรงกลาง



รูปที่ 2.10 แสดงการกำหนดจุดตัดด้วย Histogram